Triggers:

Trigger is a named database object associated with a table that activates when a particular event occurs

SYNTAX

Create trigger triggername triggertime
triggerevent on tablename for each row
Begin
≡
End;

trigger name

It is a proper name given to the trigger i.e begin created.

trigger time

It takes 2 values one is before and 2nd is after.

trigger event:

It can take any one of the following values

1) Insert
2) Delete
3) Update

* And a trigger can be invoked by one event only.

NOTE :-

A trigger must always be associated with a table. Without a table a trigger cannot exist and you have have to specify the table name after the keyword ON and you can place the SQL statements between begin and end of the trigger. This is where the actual logic of the trigger is written

Variable declaration : @ varName

Declare

old val (existing data) = old.val

new val (newly inserting data = new.val

ex : old.age new.age

We use delimiter like $$ or # to tell that the trigger start and ends there can be many ; in btw begin and end. To avoid incomplete end to the process

Q) Write a trigger to copy the content deleted from one table into another table.

```
emp ( eid, ename);
empl ( eid, ename);
delimiter $$
create trigger ABC after
  delete
    on emp
    For each row
Begin insert into empl values ( old.eid, old.ename);
end; $$
```

Trigger:

Write a trigger on accounts table for each row set the amount value as 0 if the entered amount is <0. else set the amount value is 100. if the amount entered is >100.

```
delimiter $$
  create trigger xyz before
   update
   on accounts
   For each Row
   Begin
    if new.amt<0 then set new.amt=0;
    else
    if new.amt>100
    then set new.amt=100
    end if;
    end;
    $$
```

11/11/25

Emp (eno, eage, esal);

Write a Trigger to check value of age if negative make it as positive and insert the record into the schema.

Syntax create Trigger Trigger_Name TriggerTime (Before / After)
TriggerEvent (update / delete / insert)
ON TableName
For each row
Begin

Ends.

create Trigger T
Insert
ON Emp
(

-SOLUTION-

```
delimiter $$
create Trigger std1 Before
  Insert ON Emp
For each row
Begin
if (new.eage <0)
then set new.age = 10;
end if;
End;
$$
```

Write a Triggers on students Table with sno, sname attributes and check that name of student is always entered with start with capital letter.

```
delimiter $$
create Trigger std Before
Insert on student
For each row
Begin

set new.ename = ucase(ename);

End;
$$
```

Procedures :-

Procedure is a sub program which performs a specific task. We can work with sql statements within a procedure and you must define a procedure before invoking it.

- Advantages of working with procedures :-

1. Reusability
2. Performance Improvement (↑)
3. Storage Space reduces
4. Maintainance reduces.

Syntax :-

```
delimiter $$
create procedure Proc-Name ( Parameter list )
Begin

End
$$
```

or

```
Proc-Name ( )
```

only 3 parameters ③ IN, OUT, INOUT

Para-

- The parameter list in procedure can be any one of the following : IN , OUT , INOUT .

* IN :- when you define an In parameter in a procedure then the calling program has to pass an argument to the procedure and the value of In parameter is protected.

* OUT :- the value of an OUT parameter can be changed inside the procedure and the updated value is passed back to the calling program.

* InOut :- it is a combination of In and out parameters.

Consider Emp Table.

Create a procedure to display complete details of employees.

```
delimiter $$
create Procedure get_info ( )
Begin
    select * from Emp;
End
$$
delimiter ;
call get_info ( );
```

(or)

```
call get_info ( );
$$
```

Write a procedure to display the details of employees with a given empno

```
delimiter $$
create Procedure get_info ( IN eid int )
Begin
    select * from Emp where eno = eid;
end
$$
```

```
call get_info (10);
$$
```

Write a procedure to count no-of-rows in a table based on a particular input given by user and display count.

```
delimiter $$
create Procedure P ( IN sal int , OUT count int)
Begin
Select count (*) into count from Emp where esal = sal ;
Group By (esal)
Having esal > 50000;
End;
$$
call get_info ( 50000, @b);
select @b as result;
```

→ converts to variable

→ take a variable for output parameter.

Write a procedure to display distinct sailor names for a given age entered by the user

```
delimiter $$
create Procedure display ( IN a int , OUT name varchar (20))
Begin
select DISTINCT (sname) into name from Sailors
where age = a ;
End;
$$
call display (18, @a);
select @a as result;
```

Write a procedure to find sum of 3 numbers and display the result.

```
create Procedure Sum ( IN a int, IN b. int , IN c int, OUT SUM int)
Begin
Select set, sum = a+b+c;
End a+b+c;
End $$
```

call SUM(10, 20, 30, @s);
select @s as result;

Write a factorial of a no. using procedure.

```
delimiter $$
create Procedure fact ( IN number int,
                        OUT factorial int )
Begin
Declare number, factorial; set Factorial = 1;
while ( number > 0 )
set factorial = number * number-1;
number--;
End loop;
End:
call fact ( 5, @f )
select @f as result;
```

✓

```
delimiter $$
create procedure fact ( IN n int, out factorial int)
Begin
set factorial = 1;
if ( n<=1)
then set factorial=1;
End if ;
while (n>1)
do set factorial = factorial * n;
Set n= n-1;
end while ;
End; $$
call fact ( 5, @f ) $$
Select @g as factorial $$
```

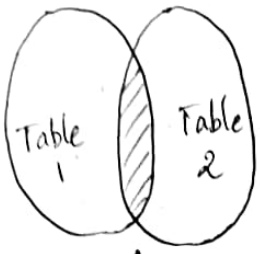
* JOINS :-

JOIN : a join is used to combine rows from 2 or more tables based on a related column between them. This is important for querying relational databases when data is distributed among multiple relations.

The types of JOINS are :

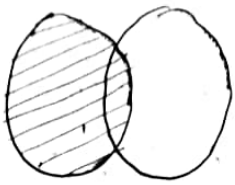
- Inner Join
- Left JOIN } outer
- Right Join }
- Full outer Join
- self Join
- Cross Join

JOINS



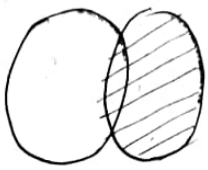
A

Inner Join



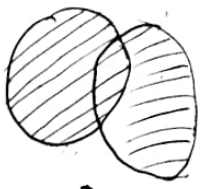
B

Left - Join



C

Right - Join



D

Cross Join

1) Inner JOIN :-

→ returns the records that have matching values in both tables.

2) Left JOIN :-

→ returns all records from left table along with matched records from right table.

T_1 = 5 tuples T_2 = 3 rows

T_1	T_2
1	1
2	2
3	3
4	NULL
5	NULL

3) Right JOIN :-

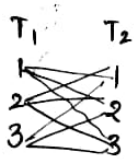
→ returns all records from right table along with the matched records from left table

T_1 = 2 rows T_2 = 3 rows

T_1	T_2
1	1
2	2
NULL	3

4) Cross JOIN :-

→ returns the cartesian product. Every row from first table is combined with every row from second table.



(9) rows.

5) Full JOIN : (FULL OUTER JOIN).

→ My SQL doesnot support full outer join but you can work with it using the union of Left JOIN and right join.

SYNTAXES :-

INNER JOIN → Select set_of_columns
from table1 INNER JOIN
table2 ON condition → tab1.col1 = tab2.col2;

Consider the relation employee with eid, ename, did as its attributes and dept with did, dname as its attributes Write a query which returns only matching rows b/w tables with respective did.

Select eid, ename ~~did~~ from employee
INNER JOIN dept ON employee.did = dept.did;

Get the employee details who works in CSE dept.

Select *, ename from employee e
INNER JOIN dept d ON employee.did = dept.did
where d.dname = "CSE";

Add salary column to the employee table and write a query to get avg salary per dept

Select eid, avg(sal) from employee e
INNER JOIN dept d ON e.did = d.did
GROUP BY ~~deg~~ did;

Write a query to show all employees and their departments including those without a department
Emp (Eid, ename, did, sal) Dept (Did, dname, loc)

Select * from Emp
left JOIN dept on emp.did = dept.did;
(can be null)

104) Select e.ename, d.dname from Emp e
Left Join dept d on e.did = d.did;

7) Write a query to list the employees who have not been assigned any department.

```
Select  e.ename, d.dname
from    Emp e   Left Join
dept d  on  e.did = d.did
and  e.ename d.dname = ~~is Null (d.dname);~~
where   d.dname is Null;
```

8) Write a query to find the departments which donot have any employees.

```
Select  ~~e.ename~~, d.dname
from    ~~Emp e~~  dept d  Left Join
Emp e   on  d.did = e.did
where   e.eid is Null;
```

(or) Select d.dname from Emp e Right Join dept d on e.did = d.did where e.eid is Null;

9) Consider the 2 tables → table1, table2. Find the union of those two tables

```
Select * from tab1
UNION  // eliminates duplicates.
Select * from tab2 ;
```

UNION → eliminates duplicates
UNION All → allows duplicates
Intersection → same as Inner JOIN.
Difference